

Current Uplink–Downlink Method Report for Finite-Blocklength Latency Experiments

June 22, 2026

Abstract

This report is a dated rewrite of the legacy combined uplink/downlink document. Its purpose is not to restate every exploratory derivation, but to document the current cleaned method set, explain how each method trains and tests, and summarize only the artifacts that are actually present today. Where a method implementation exists but a finalized comparable result artifact is missing, the method is still documented and the missing status is stated explicitly rather than replaced by guessed numbers.

1 Scope and Artifact Policy

The cleaned experiment folder now contains six main method paths:

1. uplink convergence-per-sweep baseline,
2. uplink Monte Carlo precoder net,
3. uplink Monte Carlo shared-beam precoder net,
4. downlink convergence-per-sweep baseline,
5. downlink Monte Carlo precoder net, and
6. downlink Monte Carlo shared-beam precoder net.

This rewrite follows one rule:

Use current artifacts where they exist; otherwise document the method and state that the matching current artifact is missing or incomplete.

That rule matters because the legacy report folder still contains older illustrative figures, but it does not contain a complete, current, cleaned result set for every method. In particular:

- the current cleaned uplink convergence baseline artifacts are missing from the report workspace,
- the current cleaned shared-beam report artifacts are not finalized in a comparable form, and
- the current per- $n_{k,\ell}$ Monte Carlo artifacts are available and can be summarized quantitatively.

2 Shared Problem View

Across both links, user k carries payload B_k , target reliability ε_k , power budget P_k , symbol rate $f_{s,k}$, and coherence-limited maximum blocklength T_k . If the payload is transmitted over scheduled blocks indexed by ℓ , then the latency of user k is

$$\tau_k = \frac{1}{f_{s,k}} \sum_{\ell} n_{k,\ell}. \quad (1)$$

All methods therefore try to reduce total latency through better choices of

$$\{B_{k,\ell}, n_{k,\ell}, \mathbf{F}_{k,\ell}\}, \quad (2)$$

subject to finite-blocklength feasibility

$$\frac{B_{k,\ell}}{n_{k,\ell}} \leq R_{k,\ell}^{(\text{UL/DL})}. \quad (3)$$

The main difference between the methods is not the latency definition but the *way* the precoder is produced:

- online convergence methods optimize the precoder directly during test-time scheduling,
- Monte Carlo per- $n_{k,\ell}$ methods train a precoder net whose input includes $n_{k,\ell}$,
- Monte Carlo shared-beam methods train a precoder net once per scenario and reuse the same beam across a candidate $n_{k,\ell}$ grid.

3 Method Inventory

4 Uplink Methods

4.1 Convergence per Sweep Baseline

Method idea. This is the direct online baseline. There is no offline dataset and no trained network. For a given user and a given block, the solver alternates between improving the precoder and checking which finite-blocklength $n_{k,\ell}$ is feasible. If the remaining payload does not fit in one block, the algorithm creates another block and repeats.

Training idea. There is no training stage.

Testing idea. Testing is the full algorithm: initialize a block, converge the precoder for that block, greedily shrink $n_{k,\ell}$ while the selected bit load stays feasible, commit the block, and continue until the user payload is fully served.

Artifact status. The dated report folder still contains older uplink illustrative figures, but the current cleaned baseline result artifact that should match the cleaned experiment folder is missing here. For that reason, this report does *not* quote a new quantitative baseline table for uplink convergence, and instead documents the method only.

4.2 Monte Carlo Precoder Net

Precoder-net input. The current cleaned uplink Monte Carlo net is blocklength-aware:

$$\mathbf{F}_{k,\ell} = f_{\theta_k}(H_{k,\ell}, n_{k,\ell}, \mathbf{\Sigma}_{k,\ell}, \varepsilon_k). \quad (4)$$

Each user has its own net, but the net is shared across that user's training samples.

Table 1: Current cleaned method inventory

Link	Method	Training idea	Testing idea
Uplink	Convergence per sweep	No offline training. Optimize each user block directly at test time.	For each user, create blocks as needed, optimize the block precoder, then shrink $n_{k,\ell}$ greedily while feasibility holds.
Uplink	Monte Carlo precoder net	One net per user. Dataset samples are $(H_{k,\ell}, n_{k,\ell}, \mathbf{\Sigma}_{k,\ell}, \varepsilon_k)$. Train with a Lagrangian finite-blocklength objective.	At test time, the user net predicts a fresh precoder for each candidate $n_{k,\ell}$.
Uplink	Monte Carlo shared beam	One net per user. Dataset scenarios are $(H_{k,\ell}, \mathbf{\Sigma}_{k,\ell}, \varepsilon_k)$ with a candidate $n_{k,\ell}$ grid.	Predict one beam once, then reuse it across the $n_{k,\ell}$ search.
Downlink	Convergence per sweep	No offline training. Joint on-line block optimization over active users.	In each block, first converge the active-user precoders jointly, then allocate bits and shrink $n_{k,\ell}$.
Downlink	Monte Carlo precoder net	One net per user, trained jointly. Dataset cases are $(H_\ell, n_{k,\ell}, a_\ell, \mathbf{\Sigma}_{k,\ell}, \varepsilon_k)$.	In each block, each active user net predicts its beam using the current $n_{k,\ell}$, then the scheduler allocates bits.
Downlink	Monte Carlo shared beam	One net per user, trained jointly. Dataset cases are $(H_\ell, a_\ell, \mathbf{\Sigma}_{k,\ell}, \varepsilon_k)$ plus candidate n -levels.	Predict one beam per active user once, then reuse the same beams while testing multiple $n_{k,\ell}$ values.

Training idea. The available seed-3 artifact uses train seeds 0, 1, 2. The dataset contains 864 total training samples, with 288 samples per user and four channel blocks per seed. Each sample is one

$$(\text{seed}, \text{user}, \text{block}, n_{k,\ell})$$

item, and the cleaned report artifact states that the minimum required transmitted bits per sample is fixed to 1.

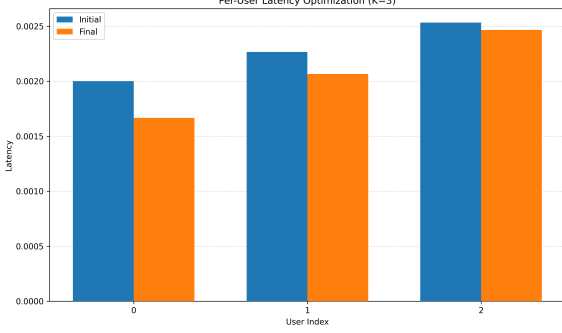
The important point is that the training loss is *not* MSE to an expert label. It is a direct Lagrangian finite-blocklength objective. The available training summary reports:

- final average user rate = 13.8813,
- final average Lagrangian = -13.8349 ,
- final average rate violation = 0.0779,
- final average power violation $\approx 10^{-6}$,
- train-eval latency reduction = 18.35% on the first training seed.

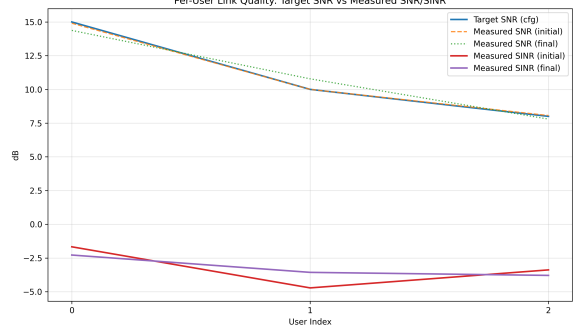
Testing idea. At test time, the scheduler still follows the greedy payload-serving logic. The difference is that the precoder itself comes from the trained user net, and a *fresh* beam is predicted for each candidate $n_{k,\ell}$ during the search.

Observed behavior from the available cleaned artifact. The seed-3 testing summary shows modest latency improvement but weaker synchronization behavior:

- total latency reduction = 8.82%,
- asynchronality reduction = -50.0% , meaning pairwise latency spread increased,
- final average SINR improved only slightly from -3.2572 dB to -3.2132 dB.



(a) Latency comparison



(b) Link-quality comparison

Figure 1: Available current uplink Monte Carlo test figures from the cleaned seed-3 artifact.

4.3 Monte Carlo Shared Beam

Precoder-net input. The shared-beam uplink net removes $n_{k,\ell}$ from the input:

$$\mathbf{F}_{k,\ell} = f_{\theta_k}(H_{k,\ell}, \mathbf{\Sigma}_{k,\ell}, \varepsilon_k). \quad (5)$$

Training idea. Each scenario contains a channel block and a candidate $n_{k,\ell}$ grid. The net predicts one beam once, and that same beam is evaluated across all candidate $n_{k,\ell}$ values. The training objective is therefore an average shared-beam finite-blocklength rate over the n -grid, rather than a fresh-beam objective at every n .

Testing idea. Testing follows the same logic: one beam prediction per scenario, then reuse of that beam while scanning candidate $n_{k,\ell}$ values until the smallest feasible value is found.

Artifact status. The method implementation exists and a shared-beam report folder was created, but this report workspace does not contain a finalized, comparable cleaned result package for the seed-3 uplink shared-beam run. Therefore the method is documented here but not included in the quantitative comparison table.

5 Downlink Methods

5.1 Convergence per Sweep Baseline

Method idea. The cleaned downlink baseline is a joint online scheduler. In each common block, the active-user set is formed from users with remaining payload. The scheduler then updates the active-user precoders jointly until the block converges, and *only after that* allocates bits and shrinks the chosen $n_{k,\ell}$.

Training idea. There is no offline training stage.

Testing idea. Testing is the algorithm itself. Two objective settings are available in the current artifacts:

- **user_rate:** a user-wise greedy objective,
- **weighted_sum_rate:** a stronger coupled objective that still uses the same online scheduling structure.

Observed behavior from available cleaned artifacts. Two current seed-3 artifacts are available:

- **user_rate:** total latency reduction = 84.46%, final average SINR = 4.73 dB,
- **weighted_sum_rate:** total latency reduction = 90.89%, final average SINR = 7.65 dB.

Among the available downlink artifacts, the weighted-sum-rate objective is the strongest baseline in this report.

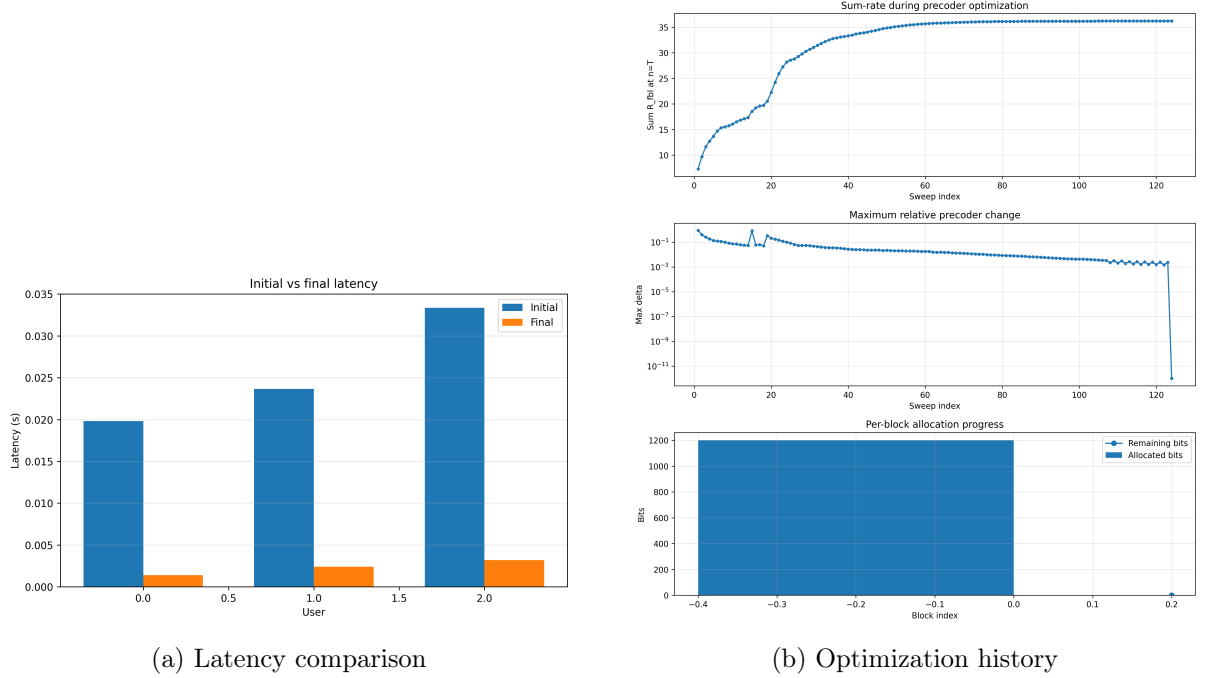


Figure 2: Available current downlink convergence-per-sweep figures for the **weighted_sum_rate** objective.

5.2 Monte Carlo Precoder Net

Precoder-net input. The current cleaned downlink Monte Carlo net is blocklength-aware and interference-aware:

$$\mathbf{F}_{k,\ell} = f_{\theta_k}(H_\ell, n_{k,\ell}, a_\ell, \Sigma_{k,\ell}, \varepsilon_k), \quad (6)$$

where H_ℓ is the full block context and a_ℓ is the active-user mask.

Training idea. The training dataset is joint rather than user-independent. One training case is a

(seed, block, active mask)

item, and the current seed-3 artifact reports:

- 720 total training cases,
- case split 240/240/240 across train seeds 0, 1, 2,
- active-user-case counts [360, 396, 432] for users 0, 1, 2.

The objective is again not MSE imitation. It is a direct Lagrangian objective built from sum finite-blocklength rate, rate feasibility, and power feasibility. The current cleaned artifact reports:

- final average training sum rate = 20.4893,
- final average user rate = 12.3861,
- final feasible training-case fraction = 1.0,
- zero final average rate violation,
- 7749 curriculum reduction events.

Testing idea. At test time, the active users in a block each predict their beam using the current $n_{k,\ell}$, active mask, and interference input. The scheduler then allocates bits and performs the usual greedy $n_{k,\ell}$ shrink rule.

Observed behavior from the available cleaned artifact. The most important finding is that stable training did *not* translate into stronger test-time scheduling than the online baseline:

- total latency changed from 0.0768 s to 0.0798 s, which is a -3.91% reduction, i.e. a regression,
- asynchronality changed by -13.79% , again a regression,
- average SINR improved only from -3.5566 dB to -2.8353 dB.

So the current downlink Monte Carlo training pipeline is numerically stable but not yet competitive with the online convergence baseline.

5.3 Monte Carlo Shared Beam

Precoder-net input. The shared-beam downlink net removes $n_{k,\ell}$ from the input:

$$\mathbf{F}_{k,\ell} = f_{\theta_k}(H_\ell, a_\ell, \mathbf{\Sigma}_{k,\ell}, \varepsilon_k). \quad (7)$$

Training idea. Each training case stores the full block context, active-user mask, and a joint n -level grid. The user nets predict one beam each, and those fixed beams are reused across the candidate $n_{k,\ell}$ levels while the objective averages the resulting sum finite-blocklength rates.

Testing idea. Testing follows the same shared-beam philosophy. For a block, the active users predict one beam each, the interference environment is evaluated with those beams fixed, and the scheduler reuses them during the greedy $n_{k,\ell}$ search.

Artifact status. The method is implemented and the report folder structure exists, but this dated rewrite does not have a finalized current cleaned shared-beam result package that is comparable to the baseline and per- n Monte Carlo seed-3 artifacts. The method is therefore documented but excluded from numerical ranking.

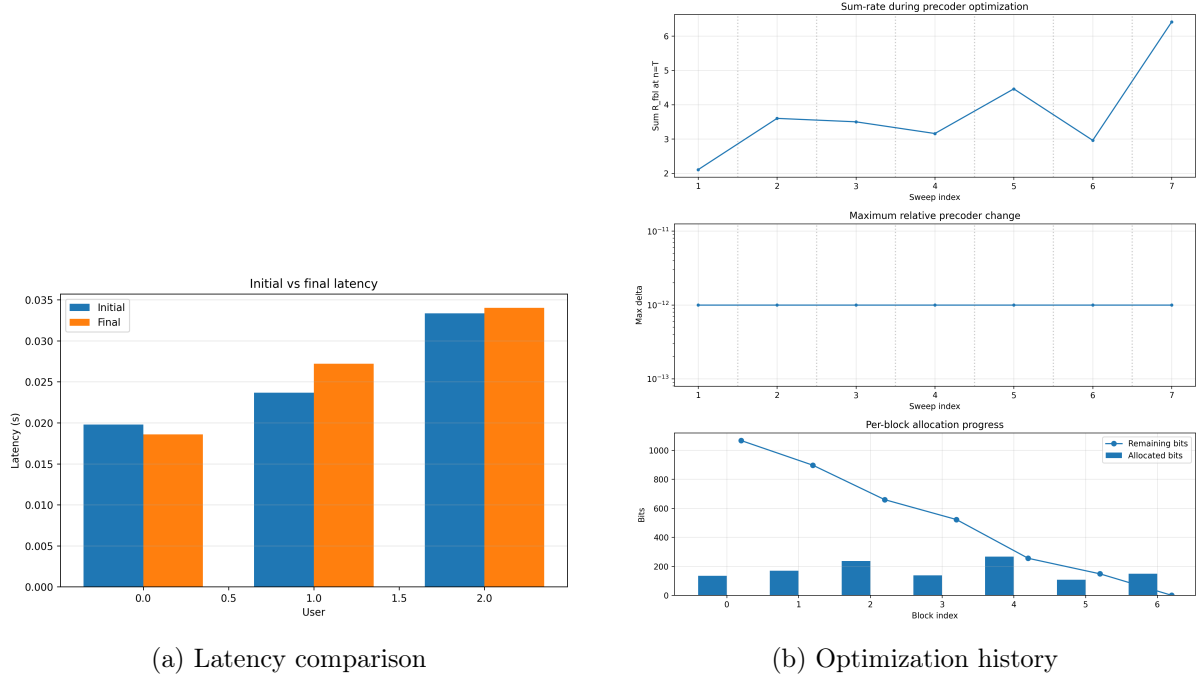


Figure 3: Available current downlink Monte Carlo test figures from the cleaned seed-3 artifact.

6 Available Quantitative Snapshot

7 What Is Missing

This dated rewrite intentionally calls out the current gaps instead of hiding them:

1. The current cleaned uplink convergence baseline result package is missing from the report workspace, so the baseline is described methodologically but not scored numerically here.
2. The shared-beam methods are implemented on both links, but the report workspace does not yet contain finalized current artifacts that are directly comparable to the available seed-3 baseline and per- n Monte Carlo runs.
3. The legacy report figures still exist in this folder, but they are not used as quantitative evidence when the matching cleaned artifact is missing.

8 Conclusion

The main change in this rewrite is the viewpoint. The report is now organized around *current methods and current artifacts*, not around older derivation branches.

The current evidence says the following:

- uplink Monte Carlo training is working as an optimization routine, but the available test artifact shows only a modest latency gain and worse asynchronality,
- downlink online convergence remains the strongest available family, especially with the `weighted_sum_rate` objective,
- downlink Monte Carlo training is stable but is not yet competitive in test-time latency,
- the shared-beam methods are important new additions to the cleaned codebase, but this report can only document their design until finalized comparable artifacts are produced.

Table 2: Method-by-method status using only currently available artifacts

Method	Artifact status	Total latency reduction	Asynchronality reduction	Comment
Uplink convergence per sweep	Missing current cleaned artifact	n/a	n/a	Method documented, but no matching current quantitative package in this workspace.
Uplink Monte Carlo precoder net	Available	8.82%	−50.0%	Modest latency gain, but pairwise synchronization worsened.
Uplink Monte Carlo shared beam	Missing finalized artifact	n/a	n/a	Implementation exists; finalized comparable report artifact not available here.
Downlink convergence (<code>user_rate</code>)	Available	84.46%	73.40%	Strong online baseline.
Downlink convergence (<code>weighted_sum_rate</code>)	Available	90.89%	86.70%	Best available downlink result in the current cleaned artifacts.
Downlink Monte Carlo precoder net	Available	−3.91%	−13.79%	Training is stable, but test-time scheduling underperforms the online baseline.
Downlink Monte Carlo shared beam	Missing finalized artifact	n/a	n/a	Implementation exists; finalized comparable report artifact not available here.

That makes this report useful in two ways: it records the exact training and testing ideas for every current method, and it also identifies which result packages still need to be generated before the full method comparison is complete.

Table 3: Available Monte Carlo training snapshots

Method	Dataset size	Final training metric	Interpretation
Uplink Monte Carlo precoder net	864 samples total, 288 per user	Avg. user rate = 13.8813, avg. Lagrangian = -13.8349	User-wise offline training works numerically and improves train-eval latency, but test gains are limited.
Downlink Monte Carlo precoder net	720 training cases, active-user-case counts [360, 396, 432]	Avg. sum rate = 20.4893, feasible fraction = 1.0	Joint training is numerically stable and fully feasible on the training set, but the learned scheduler does not yet beat the online baseline.